LIVRET DU TUTEUR

Suivi d'incidents sur un réseau backbone

Apprendre Laravel, MySQL et le développement web
en construisant un projet concret, en 8 semaines.

Semaine 1	Cadrage métier et environnement
Semaine 2	La base de données MySQL
Semaine 3	Blade, Bootstrap et authentification
Semaine 4	Inventaire réseau (CRUD)
Semaine 5	Le cœur de l'application — les incidents
Semaine 6	Journal de suivi, recherche et export
Semaine 7	Tableau de bord, rapport hebdomadaire et rôles
Semaine 8	Tests, finitions et soutenance

Laravel · Blade · Bootstrap 5 · MySQL

LIVRET DU TUTEUR

Guide du tuteur

Ce livret accompagne les 8 fiches apprenant. L'apprenant construit, semaine après semaine, l'application de suivi d'incidents backbone dont une version complète existe déjà : **le projet `suivi-incident` est le corrigé**.

Objectifs du parcours

À la fin des 8 semaines, l'apprenant doit savoir :

- **MySQL** : modéliser une base relationnelle, écrire des requêtes avec jointures et agrégats, lire un `EXPLAIN` et choisir un index ;
- **Laravel** : migrations, Eloquent et relations, routes, contrôleurs, Blade, validation, autorisations, tests ;
- **Développement** : Git, sécurité web de base (CSRF, XSS, mass assignment), découpage du code, documentation, présentation de son travail.

Contenu de chaque fiche tuteur

Rubrique	Usage
Intentions pédagogiques	Ce qui compte vraiment cette semaine
Déroulé conseillé	Un découpage jour par jour, à adapter
Corrigé	Fichiers de référence dans le projet, extraits de code, réponses SQL
Erreurs fréquentes	Symptômes et pistes à donner, sans donner la solution
Questions pour le point	Questions orales avec la réponse attendue
Grille d'évaluation	Notation sur 20
Adapter le rythme	Prolongements si l'apprenant est en avance, allègements s'il est en difficulté

Le point hebdomadaire (30 minutes, chaque vendredi)

1. **Démonstration** (10 min) : l'apprenant montre ce qui fonctionne, dans le navigateur et dans MySQL.
2. **Revue de code** (10 min) : parcourir ensemble un ou deux fichiers et les commits de la semaine.
3. **Questions** (5 min) : 2 ou 3 questions de la fiche ; l'apprenant répond sans ses notes.
4. **Bilan** (5 min) : auto-évaluation de l'apprenant comparée à votre grille, objectifs reportés.

Conseils d'accompagnement

- **Ne donnez pas le corrigé** d'un bloc ; montrez au plus un fichier pour débloquer un point précis, puis demandez à l'apprenant de le réexpliquer.

- **Faites verbaliser** : « Qu'est-ce que tu as essayé ? Que dit exactement le message d'erreur ? ». C'est la compétence la plus utile à transmettre.
- **Reliez toujours Eloquent et SQL** : « Quelle requête cette ligne envoie-t-elle à MySQL ? ».
- **Valorisez les commits réguliers** plus que la quantité de code.
- Une semaine non terminée n'est pas un échec : reportez ce qui manque en début de semaine suivante. Les rubriques « Adapter le rythme » indiquent ce qui peut être allégé.

Comptes de démonstration du corrigé

`php artisan migrate --seed` crée un administrateur `admin@backbone.local` et quatre techniciens `tech1@backbone.local` à `tech4@backbone.local`, tous avec le mot de passe `password`.

FICHE TUTEUR · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 1

Cadrage métier et environnement

Durée : 5 jours Métier NOC · Outils du développeur · Git · Modélisation

Intentions pédagogiques

Cette semaine pose les fondations : **comprendre le besoin** avant de coder. Le piège classique d'un stagiaire est de foncer dans le code. Insistez sur le schéma : une bonne modélisation évite des semaines de corrections.

L'installation peut prendre du temps (droits, versions de PHP, mot de passe MySQL). C'est normal et formateur : laissez-le chercher, mais pas plus d'une demi-journée bloqué.

Déroulé conseillé

Jour	Activité
J1	Présentation du NOC (idéalement, visite ou capture d'écran de l'outil de supervision réel). Installation des outils.
J2	Création du projet, base MySQL, utilisateur dédié, `.env`, premier `migrate`. Premier commit.
J3	Découverte de la structure Laravel : suivre une requête de la route à la vue (page d'accueil).
J4	Modélisation sur papier. **Point intermédiaire de 15 min** pour corriger le schéma avant qu'il ne parte sur une mauvaise piste.
J5	Finalisation du schéma, liste des écrans, point hebdomadaire.

Corrigé : le modèle de données attendu

```
users (id, name, email, password, role, telephone)
  │1
  ├──N incidents.declare_par_id
  ├──N incidents.assigne_a_id     (nullable)
  └──N suivis.user_id

sites (id, code UNIQUE, nom, ville, type, adresse)
  │1
  ├──N equipements.site_id
  ├──N liaisons.site_a_id
  ├──N liaisons.site_b_id
  └──N incidents.site_id

equipements (id, site_id, hostname UNIQUE, type, constructeur, modele, ip_management, statut)
  └──1─N incidents.equipement_id  (nullable)

liaisons (id, code UNIQUE, site_a_id, site_b_id, type, capacite, operateur, statut)
  └──1─N incidents.liaison_id     (nullable)

incidents (id, reference UNIQUE, titre, description, severite, statut, categorie,
           site_id, equipement_id?, liaison_id?, declare_par_id, assigne_a_id?,
           debut_at, resolu_at?, impact?, cause_racine?, solution?)
  └──1─N suivis.incident_id

suivis (id, incident_id, user_id, type, message, created_at)
```

Points à vérifier sur le schéma de l'apprenant :

- **deux relations** entre `users` et `incidents` (déclarant / assigné) ;
- **deux relations** entre `sites` et `liaisons` (A / B) ;
- `equipement_id` et `liaison_id` **facultatifs** : un incident d'énergie concerne tout un site ;
- `site_id` sur l'incident même quand l'équipement est connu (dénormalisation assumée : simplifie filtres et statistiques) ;
- la **durée n'est pas stockée** : elle se calcule à partir de `debut_at` et `resolu_at`.

Liste d'écrans attendue (ou proche) : connexion, tableau de bord, liste des incidents, création / modification d'incident, détail d'un incident avec journal, sites, équipements, liaisons, rapport hebdomadaire, gestion des utilisateurs.

Erreurs fréquentes

Erreur	Comment la repérer	Piste à donner
`Access denied for user 'root'`	`php artisan migrate` échoue	Vérifier `DB_USERNAME` / `DB_PASSWORD` ; mot de passe avec `#` → le mettre entre guillemets
`could not find driver`	Erreur PDO	Extension `pdo_mysql` absente de PHP
`.env` commité	`git log --stat`	Le retirer : `git rm --cached .env`
Une table « incident_equipement »	Schéma	Un incident concerne **un** équipement au plus : pas de table pivot
Durée stockée en colonne	Schéma	Donnée calculable = risque d'incohérence
Statut sans liste fermée	Schéma	Préparer l'idée d'enum de la semaine 2

Questions pour le point hebdomadaire

1. **Quelle différence entre P1 et P2 ? Donne un exemple réseau de chaque.** *Attendu :* P1 = coupure franche avec impact client (fibre coupée sans redondance, ville isolée). P2 = service dégradé ou redondance perdue (un des deux liens d'un anneau coupé).
2. **Comment calcule-t-on le MTTR ? Pourquoi un NOC le suit-il ?** *Attendu :* moyenne des durées début → rétablissement des incidents résolus. Mesure l'efficacité de l'équipe, sert d'engagement contractuel (SLA).
3. **Pourquoi ne pas utiliser `root` dans `.env` ?** *Attendu :* principe du moindre privilège ; si l'application est compromise, l'attaquant n'a accès qu'à sa base.
4. **Que se passe-t-il quand tu tapes une URL dans le navigateur, côté Laravel ?** *Attendu :* `public/index.php` → routes → contrôleur → (modèle) → vue → réponse HTML.
5. **Pourquoi un incident a-t-il deux liens vers `users` ?** *Attendu :* déclarant et technicien assigné sont deux rôles différents, souvent deux personnes différentes.

Grille d'évaluation (sur 20)

Critère	Points
Environnement fonctionnel (Laravel + MySQL, utilisateur dédié)	4
Git utilisé correctement (commits, `.env` exclu)	2
Schéma : 6 tables, attributs pertinents	4
Schéma : relations et cardinalités justes (doubles relations, nullables)	4
Liste des écrans cohérente avec le besoin	2
Compréhension métier à l'oral (P1/P2/P3, cycle de vie, MTTR)	4

Adapter le rythme

- **En avance** : lui faire lire le chapitre « Eloquent: Getting Started » de la documentation et écrire les migrations `sites` et `equipements`.

- **En difficulté** : fournir le schéma corrigé ci-dessus et lui demander de l'**expliquer** table par table à l'oral ; c'est la compréhension qui compte.

FICHE TUTEUR · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 2

La base de données MySQL

Durée : 5 jours SQL Migrations Eloquent Relations Enums PHP

Intentions pédagogiques

Deux compétences en parallèle : **SQL brut** (indispensable même avec un ORM) et **Eloquent**. Faites systématiquement le lien : « cette ligne Eloquent, quelle requête SQL produit-elle ? ». Un développeur qui ne sait pas ce que son ORM envoie à la base écrira des applications lentes.

Le choix `restrict` / `null` / `cascade` des clés étrangères est le moment fort de la semaine : c'est un **choix métier**, faites-le argumenter.

Déroulé conseillé

Jour	Activité
J1	Migrations `users` (role, telephone), `sites`, `equipements`. `SHOW CREATE TABLE` pour comparer.
J2	Migrations `liaisons`, `incidents`, `suivis`. Discussion sur les clés étrangères.
J3	Enums et modèles avec relations. Tests dans Tinker.
J4	Factories et seeder. `migrate:fresh --seed` doit tourner sans erreur.
J5	Exercices SQL, point hebdomadaire.

Corrigé : fichiers de référence

- Migrations : `database/migrations/2026_10_29_00000*_create_*.php`
- Enums : `app/Enums/*.php`
- Modèles : `app/Models/*.php`
- Seeder : `database/seeders/DatabaseSeeder.php`

Choix de suppression du corrigé :

Clé étrangère	Choix	Justification
`equipements.site_id`	restrict	On ne perd pas un inventaire par erreur
`liaisons.site_a_id / site_b_id`	restrict	Idem
`incidents.site_id`	restrict	L'historique des incidents doit rester
`incidents.equipement_id / liaison_id`	null	L'équipement peut être décommissionné, l'incident reste
`incidents.assigne_a_id`	null	Un technicien qui part ne supprime pas ses incidents
`incidents.declare_par_id`	restrict	Traçabilité : on sait toujours qui a déclaré
`suivis.incident_id`	cascade	Un suivi n'a pas de sens sans son incident

Génération de la référence (bonus) :

```
protected static function booted(): void
{
    static::created(function (Incident $incident) {
        $incident->reference = sprintf('INC-%s-%05d', $incident->debut_at->format('Y'),
$incident->id);
        $incident->saveQuietly();
    });
}
```

Pourquoi après l'insertion ? L'id n'est connu qu'une fois la ligne insérée (AUTO_INCREMENT). Calculer « le nombre d'incidents + 1 » avant l'insertion crée des doublons si deux techniciens déclarent en même temps.

> **Piège du seeder** — Le seeder par défaut utilise le trait `WithoutModelEvents`, qui désactive l'événement `created` : toutes les références restent vides. Il faut retirer ce trait.

Corrigé des exercices SQL

```sql
-- 1. Sites triés par ville
SELECT * FROM sites ORDER BY ville;

-- 2. Équipements Cisco
SELECT hostname, ip_management FROM equipements WHERE constructeur = 'Cisco';

-- 3. Nombre d'équipements par type
SELECT type, COUNT(*) AS total FROM equipements GROUP BY type ORDER BY total DESC;

-- 4. Équipements avec le nom de leur site
SELECT e.hostname, e.type, s.nom AS site
FROM equipements e
JOIN sites s ON s.id = e.site_id;

-- 5. Liaisons avec leurs deux extrémités (double JOIN avec alias)
SELECT l.code, sa.code AS site_a, sb.code AS site_b, l.capacite
FROM liaisons l
JOIN sites sa ON sa.id = l.site_a_id
JOIN sites sb ON sb.id = l.site_b_id;

-- 6. Suppression refusée
DELETE FROM sites WHERE code = 'BKO-DC1';
-- ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails
```

Erreurs fréquentes

Erreur	Symptôme	Piste
Ordre des migrations	`Failed to open the referenced table 'sites'`	Le nom de fichier (horodatage) fixe l'ordre : `sites` avant `equipements`
`$fillable` oublié	`Add [code] to fillable property`	Expliquer le mass assignment
Relation mal nommée	`$liaison->siteA` renvoie `null`	Préciser la clé : `belongsTo(Site::class, 'site_a_id')`
Enum non casté	`$incident->severite->label()` → « Call to a member function on string »	Ajouter le cast dans `casts()`
Références vides	`reference` NULL partout	Trait `WithoutModelEvents` dans le seeder
`push()` modifie la collection d'origine	L'admin se retrouve dans la liste des techniciens	Utiliser `concat()` qui renvoie une nouvelle collection

Questions pour le point hebdomadaire

1. **Pourquoi `cascade` sur `suivis` mais `restrict` sur `equipements.site_id` ?** *Attendu :* un suivi n'existe pas sans incident ; un site avec des équipements actifs ne doit pas disparaître par erreur.

2. **Quelle requête SQL produit `Site::withCount('equipements')->get()` ?** *Attendu :* un `SELECT` avec une sous-requête `(SELECT COUNT(*) FROM equipements WHERE sites.id = equipements.site_id) AS equipements_count`.
3. **Pourquoi utiliser un enum plutôt qu'une simple chaîne ?** *Attendu :* liste fermée, fautes de frappe impossibles, libellés et couleurs centralisés, autocomplétion dans l'éditeur.
4. **À quoi sert un index ? Pourquoi ne pas indexer toutes les colonnes ?** *Attendu :* accélère les recherches ; mais ralentit les écritures et occupe de la place.
5. **Différence entre une factory et un seeder ?** *Attendu :* la factory sait fabriquer un modèle (données aléatoires), le seeder décide quoi créer pour la démo.

Grille d'évaluation (sur 20)

Critère	Points
Migrations complètes, types et contraintes justes	4
Clés étrangères avec comportements justifiés	3
Modèles, `#[Fillable]`, toutes les relations fonctionnelles	4
Enums avec `label()` / `color()` et casts	2
Seeder réaliste, `migrate:fresh --seed` sans erreur	3
Exercices SQL (6 requêtes justes)	4

Adapter le rythme

- **En avance** : lui faire écrire le scope `scopeActifs` et l'événement de génération de référence.
- **En difficulté** : fournir la migration `incidents` et l'enum `Severite` comme modèles ; il écrit les autres par analogie. Les exercices SQL 1 à 4 suffisent.

FICHE TUTEUR · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 3

Blade, Bootstrap et authentification

Durée : 5 jours Routes Contrôleurs Blade Bootstrap Sessions Sécurité

Intentions pédagogiques

L'apprenant voit enfin quelque chose s'afficher : la motivation remonte. Profitez-en pour introduire la **sécurité** dès maintenant (CSRF, échappement, sessions) : c'est plus facile que de corriger de mauvaises habitudes plus tard.

L'authentification est volontairement écrite **à la main** (sans Breeze) : 3 méthodes, 20 lignes. L'objectif est qu'il comprenne ce qu'un starter kit fait à sa place.

Déroulé conseillé

Jour	Activité
J1	Routes, contrôleurs, `route:list`. Layout Blade minimal avec Bootstrap CDN.
J2	Menu complet, page active, zone d'alertes flash. Composant `<x-badge>`.
J3	`LoginController` : formulaire et connexion.
J4	Déconnexion, groupes `auth` / `guest`, throttle. Exercice sur la table `sessions`.
J5	Responsive, finitions, point hebdomadaire.

Corrigé : fichiers de référence

- `app/Http/Controllers/Auth/LoginController.php`
- `routes/web.php`
- `resources/views/layouts/app.blade.php`, `resources/views/auth/login.blade.php`
- `resources/views/components/badge.blade.php`

Organisation des routes attendue :

```php
Route::middleware('guest')->group(function () {
    Route::get('/login', [LoginController::class, 'create'])->name('login');
    Route::post('/login', [LoginController::class, 'store'])->middleware('throttle:5,1');
});

Route::middleware('auth')->group(function () {
    Route::post('/logout', [LoginController::class, 'destroy'])->name('logout');
    Route::get('/', DashboardController::class)->name('dashboard');
    // ... toutes les autres routes
});
```

Déconnexion complète :

```php
Auth::logout();
$request->session()->invalidate();
$request->session()->regenerateToken();
```

Page active dans le menu : `@if(request()->routeIs('incidents.*')) active @endif`.

Corrigé de l'exercice MySQL

- Connecté : une ligne dans `sessions` avec son `user_id`.
- Déconnecté : la session est invalidée ; une nouvelle session **anonyme** (user_id NULL) apparaît pour la page de connexion.
- `users.password` contient un **hash bcrypt** (`$2y$12$…`) : fonction à sens unique, on ne peut que comparer, pas déchiffrer. Si la base fuit, les mots de passe ne sont pas lisibles directement.

Erreurs fréquentes

Erreur	Symptôme	Piste
`@csrf` oublié	« 419 Page Expired »	Chaque formulaire POST/PUT/DELETE
Déconnexion par un lien `<a>`	Fonctionne mais vulnérable	Un GET ne doit jamais modifier l'état
Route `login` sans nom	`Route [login] not defined` lors d'une redirection	Le middleware `auth` cherche la route nommée `login`
Bootstrap JS absent	Le menu déroulant ne s'ouvre pas	Charger `bootstrap.bundle.min.js` en bas de page
`{!! $titre !!}`	Faille XSS	Démontrer avec un titre `<script>alert(1)</script>`
Mot de passe comparé en clair	Code avec `where('password', ...)`	Utiliser `Auth::attempt`

Questions pour le point hebdomadaire

1. **À quoi sert le jeton CSRF ? Montre-le dans le code source de la page.** *Attendu :* empêcher un site tiers de soumettre un formulaire à la place de l'utilisateur connecté. Champ caché `_token`.
2. **Pourquoi régénérer la session à la connexion ?** *Attendu :* éviter la fixation de session (un attaquant qui connaîtrait l'identifiant de session avant connexion).
3. **Quelle différence entre `{{ }}` et `{!! !!}` ?** *Attendu :* le premier échappe le HTML (sûr), le second l'affiche brut (risque XSS).
4. **Que fait `redirect()->intended()` ?** *Attendu :* renvoie vers la page initialement demandée avant la redirection vers la connexion.
5. **Comment fonctionne `throttle:5,1` ?** *Attendu :* 5 requêtes par minute ; au-delà, réponse 429.

Grille d'évaluation (sur 20)

Critère	Points
Layout commun propre, menu avec page active, responsive	4
Connexion fonctionnelle avec erreurs en français	4
Déconnexion en POST, session invalidée	2
Routes protégées (`auth` / `guest`)	3
Throttle sur la connexion	1
Composant `<x-badge>` réutilisable, messages flash	2
Compréhension sécurité à l'oral (CSRF, XSS, session)	4

Adapter le rythme

- **En avance** : « Se souvenir de moi », puis une page « Mon profil » pour changer son mot de passe (avec vérification de l'ancien).
- **En difficulté** : fournir le layout complet ; il se concentre sur le `LoginController` et les routes.

FICHE TUTEUR · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 4

Inventaire réseau (CRUD)

Durée : 5 jours CRUD Validation Formulaires Requêtes efficaces (N+1)

Intentions pédagogiques

Trois CRUD presque identiques : c'est voulu. Le **premier** (sites) s'apprend, le **deuxième** (équipements) se consolide, le **troisième** (liaisons) doit aller vite. Si le troisième prend autant de temps que le premier, c'est un signal à creuser.

Notion clé de la semaine : le **N+1**. Montrez-le concrètement en comptant les requêtes avant / après `with()`.

Déroulé conseillé

Jour	Activité
J1	Resource controller Sites : liste et création. Validation.
J2	Modification avec formulaire partagé, suppression protégée. Traduction `lang/fr/validation.php`.
J3	Équipements : liste paginée, filtres, validation IP. Mise en évidence du N+1.
J4	Liaisons. Composant `<x-bouton-supprimer>`.
J5	Exercices SQL, point hebdomadaire.

Corrigé : fichiers de référence

- `app/Http/Controllers/SiteController.php`, `EquipementController.php`, `LiaisonController.php`
- `resources/views/sites/*`, `equipements/*`, `liaisons/*` (un seul `form.blade.php` par entité)
- `resources/views/components/bouton-supprimer.blade.php`, `champ-erreur.blade.php`
- `lang/fr/validation.php`

Suppression protégée d'un site :

```
public function destroy(Site $site): RedirectResponse
{
    if ($site->equipements()->exists() || $site->incidents()->exists()) {
        return back()->with('error', 'Impossible de supprimer un site qui possède des équipements
ou des incidents.');
    }
    $site->delete();
    return redirect()->route('sites.index')->with('success', 'Site supprimé.');
}
```

À souligner : la contrainte MySQL `restrict` protège déjà, mais l'utilisateur verrait une page d'erreur 500. Le contrôle applicatif donne un **message compréhensible**. Les deux niveaux sont complémentaires.

Validation d'une liaison :

```
'site_b_id' => ['required', 'exists:sites,id', 'different:site_a_id'],
```

Corrigé des exercices SQL

```
-- 1. Ce que génère Site::withCount('equipements')
SELECT sites.*,
       (SELECT COUNT(*) FROM equipements WHERE sites.id = equipements.site_id) AS
equipements_count
FROM sites;
-- Variante équivalente :
SELECT s.*, COUNT(e.id) AS equipements_count
FROM sites s LEFT JOIN equipements e ON e.site_id = s.id
GROUP BY s.id;

-- 2. JOIN exclut les sites sans équipement ; LEFT JOIN les garde (avec des NULL / un compte à
0).
--    Attention : COUNT(*) avec LEFT JOIN donne 1 au lieu de 0 → compter COUNT(e.id).

-- 3. Liaisons touchant BKO-DC1
SELECT l.code
FROM liaisons l
JOIN sites s ON s.code = 'BKO-DC1'
WHERE l.site_a_id = s.id OR l.site_b_id = s.id;

-- 4. Mise à jour directe
UPDATE equipements SET statut = 'maintenance' WHERE hostname = 'bkodc1-core-rtr01';
```

Erreurs fréquentes

Erreur	Symptôme	Piste
`unique` sans `ignore`	Impossible d'enregistrer un site modifié	`Rule::unique('sites')->ignore($site)`
`old()` oublié	Les champs se vident après une erreur	`old('champ', $modele->champ)`
`@method('PUT')` oublié	« The POST method is not supported »	Formulaire de modification
N+1	Page lente, des dizaines de requêtes identiques	`with('site')`
`withQueryString()` oublié	Les filtres disparaissent en page 2	Sur le paginator
`COUNT(*)` avec LEFT JOIN	Les sites vides comptent 1	`COUNT(e.id)`

Questions pour le point hebdomadaire

1. **Montre-moi combien de requêtes exécute la liste des équipements, avec et sans `with('site')`.** *Attendu :* N+1 vs 2 (ou 3 avec `withCount`). Il doit savoir le prouver (log, Debugbar).
2. **Pourquoi valider côté serveur si le champ HTML a déjà `required` ?** *Attendu :* le HTML se contourne en 2 secondes (outils du navigateur, curl). Seul le serveur fait foi.
3. **Que fait le route model binding si l'id n'existe pas ?** *Attendu :* 404 automatiquement.
4. **La contrainte MySQL `restrict` suffit-elle pour la suppression d'un site ?** *Attendu :* elle protège les données, mais produit une erreur 500 ; le contrôle dans le contrôleur affiche un message clair.
5. **Comment as-tu évité de dupliquer le formulaire ?** *Attendu :* un seul `form.blade.php`, `$modele->exists` pour choisir l'action et la méthode.

Grille d'évaluation (sur 20)

Critère	Points
CRUD Sites complet, suppression protégée avec message	4
CRUD Équipements : pagination, filtres, validation IP	4
CRUD Liaisons : sites différents, badges de statut	3
Formulaires : `old()`, erreurs inline, un seul formulaire par entité	3
Pas de N+1 (vérifié)	2
Exercices SQL	2
Commits réguliers et lisibles	2

Adapter le rythme

- **En avance** : page de détail d'un site (équipements, liaisons, derniers incidents) ou import CSV d'équipements.
- **En difficulté** : faire Sites en binôme avec lui (pair programming 1 h), puis il fait Équipements seul ; Liaisons peut glisser sur la semaine suivante.

FICHE TUTEUR · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 5

Le cœur de l'application — les incidents

Durée : 5 jours Form Request Règles métier Dates Événements Eloquent

Intentions pédagogiques

C'est la semaine la plus dense. L'enjeu n'est pas seulement de faire marcher le formulaire mais de **placer chaque règle au bon endroit** (validation, contrôleur, modèle) et de comprendre **pourquoi**.

Faites-lui tenter l'attaque par mass assignment lui-même (ajouter un champ caché via les outils du navigateur) : c'est la meilleure façon de retenir la leçon.

Déroulé conseillé

Jour	Activité
J1	`IncidentRequest` complet. Création d'un incident (formulaire + `store`).
J2	Page de détail. Méthodes `dureeMinutes()` et `dureeLisible()`.
J3	Modification, règles de résolution (dates automatiques, `required_if`).
J4	Tentative de mass assignment, impression, exercices SQL.
J5	Tests manuels de toutes les règles, point hebdomadaire.

Corrigé : fichiers de référence

- `app/Http/Requests/IncidentRequest.php`
- `app/Http/Controllers/IncidentController.php` (méthodes `store`, `update`, `avecDateResolution`)
- `app/Models/Incident.php` (`booted`, `dureeMinutes`, `dureeLisible`, `formaterDuree`)
- `resources/views/incidents/form.blade.php`, `show.blade.php`

Gestion de la date de rétablissement :

```
private function avecDateResolution(array $donnees): array
{
    $termine = StatutIncident::from($donnees['statut'])->estTermine();

    if ($termine && empty($donnees['resolu_at'])) {
        $donnees['resolu_at'] = now();
    }
    if (! $termine) {
        $donnees['resolu_at'] = null;
    }

    return $donnees;
}
```

Formatage de la durée :

```
public static function formaterDuree(int|float|null $minutes): string
{
    if ($minutes === null) {
        return '–';
    }
    $minutes = (int) round($minutes);
    $jours = intdiv($minutes, 1440);
    $heures = intdiv($minutes % 1440, 60);
    $reste = $minutes % 60;

    return trim(($jours ? "{$jours}j " : '').($heures ? "{$heures}h " : '')."{$reste}min");
}
```

Corrigé des exercices SQL

```sql
-- 1. Incidents actifs, du plus ancien au plus récent
SELECT reference, titre, statut, debut_at
FROM incidents
WHERE statut IN ('ouvert', 'en_cours')
ORDER BY debut_at;

-- 2. Durée des incidents résolus
SELECT reference, TIMESTAMPDIFF(MINUTE, debut_at, resolu_at) AS duree_min
FROM incidents
WHERE resolu_at IS NOT NULL;

-- 3. Résolus sans cause racine
SELECT reference FROM incidents
WHERE statut IN ('resolu', 'clos') AND (cause_racine IS NULL OR cause_racine = '');
-- Aucun : la Form Request l'interdit. MAIS un INSERT SQL direct le pourrait :
-- la règle vit dans l'application, pas dans la base. Bon sujet de discussion.

-- 4. Déclarant et assigné
SELECT i.reference, d.name AS declare_par, a.name AS assigne_a
FROM incidents i
JOIN users d ON d.id = i.declare_par_id
LEFT JOIN users a ON a.id = i.assigne_a_id;
-- Avec un JOIN simple sur l'assigné, les incidents non assignés disparaissent.
```

Erreurs fréquentes

Erreur	Symptôme	Piste
`$request->all()` dans `create`	Faille de mass assignment	`$request->validated()` + champs serveur
`declare_par_id` en champ caché	Usurpation possible	Le fixer côté serveur
Format `datetime-local`	Champ vide en modification	`format('Y-m-d\TH:i')`
`$incident->debut_at->format()` sur un nouvel incident	« Call to a member function on null »	Opérateur `?->`
Durée négative ou bizarre	`diffInMinutes` dans le mauvais sens	Depuis `debut_at` vers `resolu_at ?? now()`
Comparaison d'enum avec une chaîne	`$incident->statut === 'resolu'` toujours faux	Comparer à `StatutIncident::Resolu`

Questions pour le point hebdomadaire

1. **Montre-moi ta tentative de mass assignment. Pourquoi a-t-elle échoué ?** *Attendu :* `validated()` ignore les champs non déclarés dans les règles, et `declare_par_id` est forcé côté serveur.

2. **Pourquoi la durée est-elle calculée et non stockée ?** *Attendu :* elle change en permanence tant que l'incident est ouvert ; la stocker créerait une donnée fausse.
3. **Pourquoi la cause racine est-elle obligatoire à la résolution ?** *Attendu :* sans elle, on ne peut pas éviter que l'incident se reproduise (amélioration continue, analyse des récurrences).
4. **Que fait `required_if:statut,resolu,clos` ?** *Attendu :* rend le champ obligatoire uniquement si `statut` vaut `resolu` ou `clos`.
5. **Ta règle « cause racine obligatoire » protège-t-elle aussi les insertions SQL directes ?** *Attendu :* non ; elle n'existe que dans l'application. Pour une garantie en base, il faudrait une contrainte `CHECK`.

Grille d'évaluation (sur 20)

Critère	Points
Création d'incident fonctionnelle, référence générée	3
Form Request complète, messages en français	3
Règles de résolution (cause / solution, dates auto, réouverture)	4
Page de détail complète et lisible	3
Protection mass assignment (démontrée)	2
Durée calculée et bien formatée	2
Exercices SQL (dont le `LEFT JOIN`)	3

Adapter le rythme

- **En avance** : filtrage JavaScript des équipements selon le site, ou ajout d'une contrainte `CHECK` MySQL `resolu_at >= debut_at` dans une migration.
- **En difficulté** : fournir `IncidentRequest` ; il se concentre sur le contrôleur et la page de détail. Les règles 4 et 6 peuvent passer en semaine 6.

FICHE TUTEUR · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 6

Journal de suivi, recherche et export

Durée : 5 jours Relations Query scopes Pagination Index MySQL Export CSV

Intentions pédagogiques

L'application devient utilisable au quotidien. Deux notions structurantes : le **principe DRY** (un seul scope pour la liste et l'export) et la **performance MySQL** avec `EXPLAIN`. C'est souvent la première fois qu'un stagiaire « voit » ce que fait la base de données : prenez le temps.

Déroulé conseillé

Jour	Activité
J1	Modèle `Suivi`, formulaire et timeline sur la page de détail.
J2	Entrées automatiques (création, changement de statut).
J3	Scope `filtrer`, formulaire de filtres, pagination avec `withQueryString`.
J4	Export CSV. `EXPLAIN` et index.
J5	Exercices SQL, point hebdomadaire.

Corrigé : fichiers de référence

- `app/Http/Controllers/SuiviController.php`
- `app/Models/Incident.php` (`suivis`, `scopeActifs`, `scopeFiltrer`)
- `app/Http/Controllers/IncidentController.php` (`index`, `update`, `export`)
- `resources/views/incidents/index.blade.php`, `show.blade.php`

Interdire l'envoi manuel d'un « changement de statut » :

```
'type' => ['required', Rule::enum(TypeSuivi::class)->only([TypeSuivi::Commentaire,
TypeSuivi::Intervention])],
```

Détection du changement de statut :

```
$ancienStatut = $incident->statut;
$incident->update(...);

if ($incident->statut !== $ancienStatut) {
    $incident->suivis()->create([...]);
}
```

Corrigé des exercices SQL

```sql
-- 1. EXPLAIN
EXPLAIN SELECT * FROM incidents WHERE statut = 'ouvert';
-- type = ref, key = incidents_statut_index → index utilisé
EXPLAIN SELECT * FROM incidents WHERE categorie = 'energie';
-- type = ALL, key = NULL → parcours complet : categorie n'est pas indexée

-- 2. Recherche
SELECT reference, titre FROM incidents WHERE titre LIKE '%fibre%';

-- 3. Nombre de suivis par incident
SELECT i.reference, COUNT(s.id) AS nb_suivis
FROM incidents i
LEFT JOIN suivis s ON s.incident_id = i.id
GROUP BY i.id, i.reference
ORDER BY nb_suivis DESC;

-- 4. 5 derniers suivis
SELECT s.created_at, i.reference, u.name, s.type, s.message
FROM suivis s
JOIN incidents i ON i.id = s.incident_id
JOIN users u ON u.id = s.user_id
ORDER BY s.created_at DESC
LIMIT 5;

-- 5. LIMIT 15 OFFSET 30 = 3e page de 15 éléments
```

> Sur la petite base de démo, MySQL peut choisir de **ne pas** utiliser l'index même s'il existe (lire 45 lignes est plus rapide). Si c'est le cas, c'est une excellente occasion d'expliquer que l'optimiseur décide selon le volume.

Erreurs fréquentes

Erreur	Symptôme	Piste
`orWhere` non regroupé	Le filtre statut ne marche plus quand on cherche un texte	Fonction englobante ; montrer `toRawSql()`
Filtrage dupliqué dans l'export	Export et liste divergent	Réutiliser le scope
Pas de BOM	Accents cassés dans Excel	`"\xEF\xBB\xBF"`
Séparateur `,`	Tout dans une colonne sous Excel FR	Séparateur `;`
Suivis non triés	Ordre aléatoire	`->latest()` dans la relation
`$incident->statut` relu après `update`	Changement jamais détecté	Mémoriser avant la mise à jour

Questions pour le point hebdomadaire

1. **Montre-moi le SQL généré par ton scope avec une recherche texte et un statut.** *Attendu :* `... WHERE (reference LIKE ? OR titre LIKE ?) AND statut = ?` avec les parenthèses.
2. **Pourquoi l'export utilise-t-il le même scope que la liste ?** *Attendu :* une seule source de vérité ; corriger un bug une fois le corrige partout.
3. **Lis-moi ce `EXPLAIN`. Faut-il ajouter un index sur `categorie` ?** *Attendu :* lecture de `type` / `key` / `rows`. Index utile seulement si on filtre souvent par catégorie et que la table grossit.
4. **Pourquoi le journal enregistre-t-il automatiquement les changements de statut ?** *Attendu :* traçabilité ; on ne peut pas compter sur l'humain pour le noter à chaque fois.
5. **Que fait `streamDownload` par rapport à écrire un fichier puis le télécharger ?** *Attendu :* envoie au fil de l'eau, pas de fichier temporaire à nettoyer.

Grille d'évaluation (sur 20)

Critère	Points
Journal de suivi : ajout et timeline	3
Entrées automatiques (création, changement de statut)	3
Filtres combinables, regroupement correct du `orWhere`	3
Pagination qui conserve les filtres	2
Export CSV lisible dans Excel, même scope que la liste	3
Analyse `EXPLAIN` comprise	3
Exercices SQL	3

Adapter le rythme

- **En avance** : notification e-mail à l'assignation (Mailable + driver `log`), ou tri de la liste par colonne.
- **En difficulté** : limiter les filtres à statut + site ; l'export CSV peut être repoussé en semaine 8.

FICHE TUTEUR · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 7

Tableau de bord, rapport hebdomadaire et rôles

Durée : 5 jours Agrégations SQL · Collections · Dates ISO · Autorisations

Intentions pédagogiques

L'apprenant produit de la **valeur pour le management** : indicateurs et fiche hebdomadaire. C'est le moment de vérifier la **justesse des chiffres** : faites recalculer un MTTR à la main, c'est souvent là qu'on découvre des erreurs de période ou de filtre.

Côté sécurité : la leçon est « **cacher n'est pas protéger** ». Faites-lui tester l'accès direct aux URL avec un compte technicien.

Déroulé conseillé

Jour	Activité
J1	Tableau de bord : cartes par statut, sévérités, derniers incidents.
J2	MTTR, top sites, P1 en cours. Vérification à la main.
J3	Rapport hebdomadaire : sélection de semaine, indicateurs, répartitions.
J4	Gate `admin`, gestion des utilisateurs, protections.
J5	Exercices SQL, impression du rapport, point hebdomadaire.

Corrigé : fichiers de référence

- `app/Http/Controllers/DashboardController.php`, `RapportController.php`, `UserController.php`
- `app/Providers/AppServiceProvider.php` (Gate `admin`)
- `resources/views/dashboard.blade.php`, `rapports/hebdomadaire.blade.php`, `users/*`

Incidents encore ouverts en fin de semaine :

```
Incident::query()
    ->where('debut_at', '<=', $fin)
    ->where(fn ($q) => $q->whereNull('resolu_at')->orWhere('resolu_at', '>', $fin))
    ->get();
```

Protection à trois niveaux :

```
Gate::define('admin', fn (User $user) => $user->estAdmin());                    // définition
Route::resource('users', UserController::class)->middleware('can:admin');       // route
Gate::authorize('admin');                                                        //
IncidentController::destroy
```

Corrigé des exercices SQL

```sql
-- 1. Par statut
SELECT statut, COUNT(*) AS total FROM incidents GROUP BY statut;

-- 2. MTTR global (minutes)
SELECT ROUND(AVG(TIMESTAMPDIFF(MINUTE, debut_at, resolu_at))) AS mttr_min
FROM incidents WHERE resolu_at IS NOT NULL;

-- 3. MTTR par catégorie
SELECT categorie, COUNT(*) AS nb,
       ROUND(AVG(TIMESTAMPDIFF(MINUTE, debut_at, resolu_at))) AS mttr_min
FROM incidents WHERE resolu_at IS NOT NULL
GROUP BY categorie ORDER BY mttr_min DESC;

-- 4. Par semaine ISO (mode 3 = lundi, semaine ISO)
SELECT YEARWEEK(debut_at, 3) AS semaine, COUNT(*) AS total
FROM incidents GROUP BY semaine ORDER BY semaine;

-- 5. Top 3 sites en P1 sur 30 jours
SELECT s.code, s.nom, COUNT(*) AS nb_p1
FROM incidents i JOIN sites s ON s.id = i.site_id
WHERE i.severite = 'critique' AND i.debut_at >= NOW() - INTERVAL 30 DAY
GROUP BY s.id, s.code, s.nom
ORDER BY nb_p1 DESC LIMIT 3;
```

Exercice 6 : les deux MTTR diffèrent normalement. Le tableau de bord ne prend que les incidents **résolus dans les 30 derniers jours** ; la requête 2 prend **tous** les résolus. Attendu : qu'il trouve lui-même l'explication. C'est la leçon principale : **un indicateur n'a de sens qu'avec sa période et son périmètre**.

Erreurs fréquentes

Erreur	Symptôme	Piste
`endOfWeek()` sans `copy()`	Début et fin identiques, rapport vide	Carbon est mutable
Incidents ouverts inclus dans le MTTR	MTTR qui augmente en continu	Ne prendre que `resolu_at` non nul
Bouton caché sans protection serveur	Un technicien supprime via une requête forgée	`Gate::authorize` / middleware `can`
`YEARWEEK` sans mode	Semaines décalées le dimanche	`YEARWEEK(date, 3)`
SQL MySQL dans le contrôleur	Les tests SQLite plantent	Calculer en PHP ou isoler le SQL spécifique
Admin se supprime lui-même	Plus aucun admin	Contrôle `$user->is($request->user())`

Questions pour le point hebdomadaire

1. **Calcule à la main le MTTR de 3 incidents de ton rapport et compare avec l'écran.** *Attendu :* identique à la minute près.
2. **Pourquoi ton MTTR SQL global diffère-t-il de celui du tableau de bord ?** *Attendu :* période différente (30 derniers jours vs tout l'historique).
3. **Un technicien voit-il le bouton Supprimer ? Peut-il quand même supprimer ? Prouve-le.** *Attendu :* il ne le voit pas, et une requête DELETE forgée renvoie 403.
4. **Quand préférer un calcul SQL à une collection ?** *Attendu :* gros volumes → SQL ; petits volumes déjà chargés → collection, plus lisible.
5. **Comment est définie la semaine 1 en ISO ?** *Attendu :* la semaine qui contient le premier jeudi de l'année, du lundi au dimanche.

Grille d'évaluation (sur 20)

Critère	Points
Tableau de bord complet et juste	4
MTTR correct (vérifié à la main)	3
Rapport hebdomadaire : bornes de semaine et « encore ouverts » justes	4
Rapport imprimable et lisible	2
Autorisations protégées côté serveur, gestion des utilisateurs	4
Exercices SQL et analyse de l'écart de MTTR	3

Adapter le rythme

- **En avance** : graphique Chart.js des incidents par jour, ou envoi automatique du rapport chaque lundi (commande artisan + planificateur).
- **En difficulté** : fournir le calcul des bornes de semaine ; il se concentre sur les indicateurs et les autorisations.

FICHE TUTEUR · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 8

Tests, finitions et soutenance

Durée : 5 jours Tests automatisés Qualité du code Documentation Présentation

Intentions pédagogiques

Fin de parcours. Trois objectifs : **fiabiliser** (tests), **transmettre** (README, code lisible) et **convaincre** (démonstration). La soutenance est aussi l'occasion d'évaluer la progression sur l'ensemble du stage, pas seulement la semaine.

Demandez-lui de **casser volontairement** une règle métier pour voir un test échouer : c'est ce qui fait comprendre l'utilité des tests.

Déroulé conseillé

Jour	Activité
J1	Premiers tests : invités, connexion, pages accessibles.
J2	Tests métier : création, résolution, suivi, autorisations, filtres, rapport.
J3	Pint, relecture du code, correction des 5 irritants.
J4	README testé par une autre personne. Répétition de la démonstration.
J5	**Soutenance** (15 min de démo + 15 min de questions). Bilan du stage.

Corrigé : fichiers de référence

- `tests/Feature/IncidentTest.php` (10 tests, 43 assertions)
- `database/factories/*` (états `admin()`, `resolu()`)
- `README.md`

Exemple de test d'autorisation :

```
public function test_un_technicien_ne_gere_pas_les_utilisateurs_ni_ne_supprime(): void
{
    $incident = Incident::factory()->create();

    $this->actingAs(User::factory()->create());

    $this->get(route('users.index'))->assertForbidden();
    $this->delete(route('incidents.destroy', $incident))->assertForbidden();
}
```

Piège rencontré dans le corrigé : un test du rapport hebdomadaire vérifiait qu'un incident d'une autre semaine n'apparaissait pas, mais cet incident était **encore ouvert** et apparaissait donc légitimement dans « encore ouverts ». Le test était faux, pas l'application. Bon exemple à raconter : **un test qui échoue n'a pas toujours raison**.

Erreurs fréquentes

Erreur	Symptôme	Piste
`RefreshDatabase` oublié	Tests qui dépendent de l'ordre d'exécution	Base vierge à chaque test
Tests sur la base MySQL de dev	Données de démo effacées	Vérifier `phpunit.xml` (SQLite en mémoire)
SQL spécifique MySQL	Tests qui plantent sous SQLite	Calcul en PHP ou SQL portable
Dates aléatoires des factories	Tests instables (« flaky »)	Fixer `debut_at` dans le test
README non testé	Étape manquante (`key:generate` , création de base)	Le faire suivre par quelqu'un d'autre
Démo improvisée	Bugs en direct	Scénario écrit et répété deux fois

Questions de soutenance (avec réponses attendues)

1. **Pourquoi des enums plutôt que des chaînes pour statut et sévérité ?** *Attendu :* valeurs contrôlées, libellés et couleurs centralisés, erreurs détectées par l'éditeur.
2. **Que se passe-t-il si deux techniciens résolvent le même incident en même temps ?** *Attendu :* le dernier enregistrement gagne (« last write wins »). Piste : verrouillage optimiste, champ `version` . Il doit savoir **identifier** la limite.
3. **Ton application est-elle protégée contre l'injection SQL ? Comment ?** *Attendu :* Eloquent utilise des requêtes préparées ; attention à `selectRaw` / `whereRaw` avec des données utilisateur.
4. **Comment ferais-tu pour recevoir automatiquement les alarmes de la supervision ?** *Attendu :* une route API (token), validation, création d'incident ; éviter les doublons pour une même alarme.
5. **Si la table `incidents` atteint 5 millions de lignes, qu'est-ce qui ralentit en premier ?** *Attendu :* la recherche `LIKE '%…%'` , le tableau de bord (agrégations) ; pistes : index, index FULLTEXT, mise en cache des indicateurs.
6. **Qu'as-tu appris de plus important pendant ce stage ?** *Attendu :* réponse personnelle ; valoriser le recul (méthode, sécurité, SQL…), pas la liste des fonctionnalités.

Grille d'évaluation de la semaine (sur 20)

Critère	Points
Au moins 10 tests pertinents, tous verts	5
Tests lisibles (Préparer / Agir / Vérifier, factories avec états)	2
Code formaté, sans code mort ni `dd()`	2
README complet et vérifié par un tiers	3
Démonstration fluide, scénario réaliste	4
Réponses aux questions, recul sur les limites	4

Bilan du stage (grille globale)

Compétence	Acquis	En cours	Non acquis
Modéliser une base de données relationnelle	☐	☐	☐
Écrire des requêtes SQL (JOIN, GROUP BY, agrégats)	☐	☐	☐
Lire un plan d'exécution (`EXPLAIN`), choisir un index	☐	☐	☐
Migrations, modèles et relations Eloquent	☐	☐	☐
Routes, contrôleurs, vues Blade	☐	☐	☐
Validation et règles métier	☐	☐	☐
Sécurité web (CSRF, XSS, mass assignment, autorisations)	☐	☐	☐
Tests automatisés	☐	☐	☐
Git et travail incrémental	☐	☐	☐
Communication : expliquer et défendre ses choix	☐	☐	☐

Pour la suite

Les pistes d'évolution de la fiche apprenant (pièces jointes, disponibilité des liaisons, API de supervision…) peuvent servir de sujet de prolongation de stage ou de projet pour un nouvel apprenant qui reprendrait le code existant, ce qui est un excellent exercice de lecture de code.